Day 6 – Detecting Expired Keys (Management Command)

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure the virtual environment is active (`source .venv/bin/activate`).
   - Run the Django dev server (`python manage.py runserver`).
   - Open your project directory in the IDE, ready to create new nested directories and python scripts.
2. **Prerequisites Checklist**:
   - Students must have completed Day 5 successfully (Order database tables migrated, and order creation view operational).
3. **Required Material**:
   - Whiteboard or slide showing standard command line task scheduling setups (such as cron tabs or background runners).

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain Django custom management commands and their use cases.
- Scaffold the specific package structure required for custom Django commands.
- Subclass `BaseCommand` to write command line utilities.
- Perform high-performance database batch updates using the ORM `.update()` method.
- Construct timezone-aware database queries using the `__lte` (less than or equal to) lookup.
- Configure cron jobs to run custom commands on a schedule.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Define background tasks. Explain when to use management commands vs views or tasks.

2. Key Concepts & Core Ideas	15 min	Explain `BaseCommand` methods, stdout redirection, batch ORM operations, and timezone filters.
3. Live Coding Walkthrough	40 min	Create directories and files, implement `check_expired_keys` logic, run manually, and discuss cron configurations.
4. Check for Understanding	10 min	Q&A on stdout vs print, `.update()` side effects, and crontab syntax.
5. Class Wrap-up & Checkpoint	5 min	Verify expired keys are successfully processed by running the command in the terminal.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Ask students: *"So far, we have built APIs that run when a user makes a request (like buying a key). But who updates a key when it expires? Does the user have to request an endpoint to expire it? No, the system must check in the background automatically."*
- **Management Commands**: Explain that Django commands are scripts that can be invoked via `python manage.py [command]`. They are perfect for background scripts triggered by the OS scheduler (like cron).

2. Key Concepts & Core Ideas (15 min)

Introduce these command line concepts:

- **`BaseCommand`**: The base class for all Django CLI scripts. Custom commands must subclass it and implement a `handle()` method.
- **Stdout and Stderr Redirection**: Why commands must print output using `self.stdout.write()` instead of standard Python `print()`. It allows output filtering and unit testing verification.
- **`self.style.SUCCESS()`**: Colorizes terminal logs (e.g. green for success, red for errors) for better visual feedback.
- **Bulk Database Updates**: Contrast looping over matching rows and calling `.save()` (creates `N` SQL queries) with `.update()` (creates `1` SQL query).
- **Timezone-aware datetime lookups**: Why using `timezone.now()` is mandatory when querying timezone-aware fields, and how `expires_at__lte` translates to SQL `expires_at <= NOW()`.

---

3. Live Coding Walkthrough (40 min)

Step 3.1: Scaffolding the Directory Structure

- **Explain**: Django scans directories to discover custom commands. We must create a specific nested folder layout under our custom app. Emphasize that every directory must contain an empty `__init__.py` file so Python treats them as packages.
- **Code**: Create the folder tree in the terminal:

```
games/
  management/
    __init__.py
    commands/
      __init__.py
      check_expired_keys.py
```

- **Verify**: Confirm that both `__init__.py` files are created.
- ⚠ **Common Pitfalls**:
  - **Missing `__init__.py`**: If either package initialization script is missing, Django will fail to register the command, returning `Unknown command: 'check_expired_keys'`.

Step 3.2: Implementing the Command Logic

- **Explain**: Create `games/management/commands/check_expired_keys.py`. Write a command class that queries all active keys whose expiration datetime is in the past, changes their status to `expired`, and prints a count of affected keys.
- **Code**: Write the following content inside `games/management/commands/check_expired_keys.py`:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey


class Command(BaseCommand):
    help = 'Mark active keys whose expiry has passed as expired.'

    def handle(self, *args, **options):
        expired_keys = GameKey.objects.filter(
            expires_at__lte=timezone.now(),
            status='active'
        )
        count = expired_keys.update(status='expired')
        self.stdout.write(self.style.SUCCESS(f'Expired {count} keys.'))
```

- **Verify**: Ensure the file compiles without syntax errors.

Step 3.3: Manual CLI Testing

- **Explain**: We will manually trigger a key expiration. Log into the Django Admin dashboard and create or modify a `GameKey` record so that its `expires_at` value is set to one hour in the past, and its status is set to [redacted]. Then, run the command.

- **Code**: Execute the management command:

```
python manage.py check_expired_keys
```

- **Verify**: Check the console output. It should output a success message (e.g. `Expired 1 keys.`). Reload the Admin page and verify the status of the expired key has changed to `Expired`.
- ⚠ **Common Pitfalls**:
  - **Using `datetime.now()` instead of `timezone.now()`**: If you use naive Python datetimes, Django will raise a `TypeError` due to comparing offset-naive and offset-aware datetimes.
  - **Queryset caching after updates**: Remind students that `.update()` modifies the rows in the database, but does not update active Python model objects in memory. If we reference the objects inside a cached queryset after calling `.update()`, they will still show the old status.

Step 3.4: Scheduling in Production via Cron

- **Explain**: To run this script every 15 minutes automatically on our production Linux server, we would add a line to our system crontab file pointing to the virtualenv python interpreter.
- **Code**: Open crontab config (via `crontab -e`) and add:

```
*/15 * * * * /path/to/.venv/bin/python /path/to/manage.py check_expired_keys
```

- **Verify**: Discuss the cron parameters: `*/15` means every 15 minutes, followed by hour, day of month, month, and day of week wildcards.

---

4. Check for Understanding (10 min)

> ***Question***: *"Why do we use `self.stdout.write()` instead of `print()` inside a management command?"*
> ***Answer***: *Using `self.stdout.write()` is best practice because it integrates with Django's internal logging systems and output redirection. In testing, it allows the output to be intercepted and asserted on (using `call_command`), whereas `print()` writes directly to the standard system stdout, making it harder to test or suppress.*

> ***Question***: *"What's the performance difference between `.update()` and looping + `.save()`? Are there situations where you'd prefer `.save()` anyway?"*
> ***Answer***: *`.update()` executes a single SQL `UPDATE` statement in the database, which is extremely fast and efficient for bulk operations. Looping and calling `.save()` executes a separate SQL query for each record (N queries), which is slow. However, you would prefer `.save()` if you need to trigger model `save()` overrides, pre/post-save signals, or validation, which `.update()` bypasses.*

> ***Question***: *"What does `expires_at__lte=timezone.now()` translate to in SQL?"*
> ***Answer***: *It translates to a `WHERE` clause: `WHERE expires_at <= 'CURRENT_TIMESTAMP_VALUE'`. This filters for records where the expiration datetime is less than or equal to the current time.*

> ***Question***: *"If we want this command to run every 15 minutes automatically, what are our options?"*
> ***Answer***: *1) Setup a system-level cron job that executes the command through the virtualenv python. 2) Use a Celery Beat task that runs the management command (or its logic) on a schedule. 3) Use cloud-specific schedulers (like Render Cron Jobs or AWS EventBridge) to trigger the command.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - [ ] The `check_expired_keys.py` script is placed under `games/management/commands/`.
  - [ ] Creating a key with a past expiration date and running `python manage.py check_expired_keys` runs successfully.
  - [ ] The script prints a green confirmation message to the terminal.
  - [ ] The database records are updated from `active` to `expired` successfully.